PSTAT 134 Final Project

Eric Livshiz, Jake Vurpillat, Dylan Crookes, Tanveer Singh, Samuel Erlikhman

2026-06-08

Introduction

We live in the age of streaming services. In the U.S., Netflix, Prime Video, and Disney+ are the most popular, with many households often having many subscriptions. (https://www.statista.com/chart/25382/most-used-video-streaming-platforms/?srsltid=AfmBOoox1XeLW3YlbDt9wzhbtbIxPH0Cxz3AqZt9FTke_DGyZl5rTXjR). While it's nice to have options, users may start to feel choice overload with the amount of content to choose from; having too many options can result in no choice. So, a user may spend time scrolling endlessly, with no result (at least we do). So, we thought of a question which many have been thought of before; can we use data to recommend movies that better match a user's preferences?

This project utilizes recommender systems, as well as natural language processing (NLP). The recommender system is used for finding patterns within the data, and suggest similar films based on common features (i.e., a *Star Wars IV* query may return other *Star Wars* films, or *Indiana Jones* due to Harrison Ford starring in both, or *Jurassic Park*, due to John Williams composing the score of both films). Regarding NLP, a few of the features that define similarity and we intend to use are stored in text format (key plot words as an example). We can of course parse the list and analyze the words individually, but we want to preserve structural representations (think of all the famous movie quotes which will lose meaning otherwise!). We also aim to build an NLP-based chat interface, which allows the user to input a natural query (i.e., "recommend me a movie similar to *Top Gun*"), and receive a useful output.

Goal

We want to answer the following question: "Can we accurately recommend movies given film metadata, such as genre, keywords, cast, and other relevant features?", using recommender systems and NLP, as well as to create a user interface to interact with our model

Data

We are lucky that we are not the only ones who care about statistics and films! There exists plenty of datasets regarding films, and we have chosen to use The Movies Dataset from Kaggle, as it contains data for ~45,000 films (we use "film" and "movie" interchangeably throughout this write-up, and want to point this out here to avoid any confusion, let's hope our model believes there is no difference in these terms as well). The data is collected from MovieLens, a massive dataset which we will not be using, as it contains 33 million ratings and 86,000 movies, and is not reliant for research, and the details, credits, and keywords are taken from TMDB API. The dataset contains structured and unstructured data, and we will focus on the following files included:

- `movies_metadata.csv` - overall movie information, i.e., title and genre
- `keywords.csv` - quite useful, as it contains plot-related keywords which are stored as JSON objects
- `credits.csv` - self-explanatory; the information about cast and crew
- `links.csv` - mapping between the MovieLens, TMDB, and IMDB identifiers

The dataset includes two more CSVs, `links_small.csv` and `ratings_small.csv`, which are a subset of 9,000 movies - we want to work with the larger dataset to be more accurate, and we are not sure how the subset was selected (we are not sure if it is random, the first 9,000, or what algorithm what used to select the subset). It is important to note, that this dataset was last updated in 2017, meaning our dataset is missing thousands of recent movies. Looking at the above example, a natural similar film to *Top Gun* would be *Top Gun: Maverick*, which came out in 2022, but our system would fail to recommend it.

Methodology

We'd like to introduce how we will build our recommendation and NLP systems.

Recommendation System This project stems from a content-based recommender systems, meaning that movies are recommended based on their similarity to other movies. This "similarity" is taken from film characteristics, i.e., genre and cast. We will implement this idea by making each movie a feature vector based on its attributes, we will use a similarity measure to be able to compare the movies, compute similarity score, and output the best n options. This can be achieved by using cosine similarity, where $cos(\theta) = \frac{A \cdot B}{||A||||B||}$. We measure the angle between two feature vectors, where A and B are two movies. The numerator is the dot product, meaning the overlap of the films' features, and the denominator normalizes given the length of vector. The resulting value is from -1 to 1; -1 means that the films are absolute opposites, whereas 1 means that the movies are almost identical (think *Mission Impossible* films). Cosine similarity is quite useful in our context as it focuses on feature patterns, rather than the scale.

NLP Natural language processing is used in this project to understand text-based movie features, as well as for user interaction. As previously mentioned, the keywords and cast lists are stored in a text format and are not easily interpreted by a computer. One way to deal with this data is to find the most useful text-based features (genre and keyword, versus studio or release date). We merge these features into a text representation for each movie (which is referred to as a bag of words), and convert the text to numerical representation using the Term Frequency-Inverse Document Frequency (TF-IDF) method. This process assigns weights to words based on how important it is to the entire dataset $TF - IDF(t, d) = TF(t, d) \cdot log(\frac{N}{DF(t)})$. We can then feed natural sentences to a chatbot, have it process the query, and formulate it into a proper query to the recommender system. Let's go ahead and get started with the data cleaning.

Data Cleaning

Dataset Introduction

Let's start by introducing the datasets provided:

```
keywords <- read_csv("data/movies/keywords.csv")
links <- read_csv("data/movies/links.csv")
ratings <- read_csv("data/movies/ratings.csv")
movies <- read_csv("data/movies/movies_metadata.csv")
credits <- read_csv("data/movies/credits.csv")

dataset_summary <- tibble(
  dataset = c("credits", "keywords", "links", "ratings", "movies_metadata"),
  observations = c(
    nrow(keywords),
    nrow(links),
    nrow(ratings),
    nrow(movies),
```

```r
    nrow(credits)
  ),
  variables = c(
    ncol(keywords),
    ncol(links),
    ncol(ratings),
    ncol(movies),
    ncol(credits)
  )
) %>% kable()

column_summary <- tibble(
  dataset = c("credits", "keywords", "links", "ratings", "movies_metadata"),
  columns = c(
    paste(colnames(keywords), collapse = ", "),
    paste(colnames(links), collapse = ", "),
    paste(colnames(ratings), collapse = ", "),
    paste(colnames(movies), collapse = ", "),
    paste(colnames(credits), collapse = ", ")
  )
)

column_summary %>% kable()
```

dataset	columns
credits	id, keywords
keywords	movieId, imdbId, tmdbId
links	userId, movieId, rating, timestamp
ratings	adult, belongs_to_collection, budget, genres, homepage, id, imdb_id, original_language, original_title, overview, popularity, poster_path, production_companies, production_countries, release_date, revenue, runtime, spoken_languages, status, tagline, title, video, vote_average, vote_count
movies_metadata	cast, crew, id

Let's explore each dataset individually.

Individual Dataset Cleaning

Keywords:

```r
head(keywords, n = 2) %>% kable()
```

id	keywords
862	[{'id': 931, 'name': 'jealousy'}, {'id': 4290, 'name': 'toy'}, {'id': 5202, 'name': 'boy'}, {'id': 6054, 'name': 'friendship'}, {'id': 9713, 'name': 'friends'}, {'id': 9823, 'name': 'rivalry'}, {'id': 165503, 'name': 'boy next door'}, {'id': 170722, 'name': 'new toy'}, {'id': 187065, 'name': 'toy comes to life'}]
8844	[{'id': 10090, 'name': 'board game'}, {'id': 10941, 'name': 'disappearance'}, {'id': 15101, 'name': "based on children's book"}, {'id': 33467, 'name': 'new home'}, {'id': 158086, 'name': 'recluse'}, {'id': 158091, 'name': 'giant insect'}]

Here we see two columns, `id` for the movie ID, as well as a JSON string representing keywords, which convenitently have IDs as well. No need to remove anything from this data.

Links:

```
head(links, n = 2) %>% kable()
```

movieId	imdbId	tmdbId
1	0114709	862
2	0113497	8844

Here we see a map of 3 different IDs. We notice that `tmbdId` correlates with the ID from `keywords`. Eventually, we will need to merge data to be one dataset, so let's keep this dataset as well.

Ratings:

```
head(ratings, n = 2) %>% kable()
```

userId	movieId	rating	timestamp
1	110	1.0	1425941529
1	147	4.5	1425942435

Here's a new ID variable, `userId`. We also see `timestamp`, which looks like the timestamp of when the movie was rated. Not very useful for us; we know when the movie was released and don't care when the film was watched.

```
ratings_clean <- ratings %>%
  select(
    - timestamp
  )
```

Movies: For the movie metadata dataset, as it has 24 columns. We don't need to display all of them (as they will not fit in the resulting document), and not all of them are useful. Let's get rid of the less useful columns, and then display the rest:

```
movies_clean <- movies %>%
  select(
    -homepage,
    -original_title,
    -poster_path,
    -video,
    -status,
    -original_title
  )

head(movies_clean, n = 2) %>% kable()
```

adult	belongs_to_collection	budget	genres	homepage	id	imdb_id	original_language	original_title	overview	popularity	poster_path	production_companies	production_countries	release_date	revenue	runtime	spoken_languages	status	tagline	title	vote_average	vote_count
FALSE	{'id': 10194, 'name': 'Toy Story Collection', 'poster_path': '/7G9915LfUQ2lVfwMEEhDsn3kT4B.jpg', 'backdrop_path': '/9FBwqcd9IRruEDUrTdcaafOMKUq.jpg'}	30000000	[{'id': 16, 'name': 'Animation'}, {'id': 35, 'name': 'Comedy'}, {'id': 10751, 'name': 'Family'}]	http://toystory.disney.com/toy-story	862	tt0114709	en	Toy Story	Led by Woody, Andy's toys live happily in his room until Andy's birthday brings Buzz Lightyear onto the scene. Afraid of losing his place in Andy's heart, Woody plots against Buzz. But when circumstances separate Buzz and Woody from their owner, the duo eventually learns to put aside their differences.	21.946943	/rhIRbceoE9VR4IVTMES8GxrWzYq.jpg	[{'name': 'Pixar Animation Studios', 'id': 3}]	[{'iso_3166_1': 'US', 'name': 'United States of America'}]	1995-10-30	373554033	81	[{'iso_639_1': 'en', 'name': 'English'}]	Released		Toy Story	7.7	5415
FALSE		65000000	[{'id': 12, 'name': 'Adventure'}, {'id': 14, 'name': 'Fantasy'}, {'id': 10751, 'name': 'Family'}]		8844	tt0113497	en	Jumanji	When siblings Judy and Peter discover an enchanted board game that opens the door to a magical world, they unwittingly invite Alan – an adult who's been trapped inside the game for 26 years – into their living room. Alan's only hope for freedom is to finish the game, which proves risky as all three find themselves running from giant rhinoceroses, evil monkeys and other terrifying creatures.	17.015539	/vzmL6fP7aPzqkSi3ITe7kMXverb.jpg	[{'name': 'TriStar Pictures', 'id': 559}, {'name': 'Teitler Film', 'id': 2550}, {'name': 'Interscope Communications', 'id': 10201}]	[{'iso_3166_1': 'US', 'name': 'United States of America'}]	1995-12-15	962797742	104	[{'iso_639_1': 'en', 'name': 'English'}, {'iso_639_1': 'fr', 'name': 'Français'}]	Released	Welcome to the dice and unleash the excitement!	Jumanji	6.9	2413

We won't display or change the `credits` dataset, as the `cast` and `crew` columns are stored as JSON strings, and are quite lengthy.

Merging

Let's now go ahead and merge datasets. We see the following mapping: movies$id = tmdbIdkeywordsid =$ tmdbId credits$id = tmdbIdratingsmovieId =$ MovieLens Id

Links does not have IDs, but the order in which the movies are stored is the same order as the the movie metadata dataset. Let's merge movies, keywords, and credits by tmdbId, map ratings' movieId to tmdbId using links, summarize ratings by movie, and merge rating summaries into the full data.

```r
movies_full <- movies_clean %>%
  left_join(keywords, by = "id") %>%
  left_join(credits, by = "id")

ratings_mapped <- ratings_clean %>%
  left_join(links, by = "movieId")

ratings_summary <- ratings_mapped %>%
  group_by(tmdbId) %>%
  summarise(
    avg_user_rating = mean(rating, na.rm = TRUE),
    num_user_ratings = n(),
    .groups = "drop"
  )

movies_final <- movies_full %>%
  left_join(
    ratings_summary,
    by = c("id" = "tmdbId")
  )
```

Let's take a look at the column names of this final dataset which we'll use for our analysis:

```r
tibble(dataset = "movies_final",
       columns = paste(colnames(movies_final),
       collapse = ", ")) %>% kable(col.names = c("Dataset", "Columns"))
```

Dataset	Columns
movies_final	adult, belongs_to_collection, budget, genres, id, imdb_id, original_language, overview, popularity, production_companies, production_countries, release_date, revenue, runtime, spoken_languages, tagline, title, vote_average, vote_count, keywords, cast, crew, avg_user_rating, num_user_ratings

Tidying

Let's see if we can make a few columns look nicer. Taking a look at original language, we see that the character values are two-letter ISO 639-1 language codes:

```r
movies_final %>%
  distinct(original_language) %>%
  head(n = 5) %>%
  kable()
```

original_language
en
fr
zh
it
fa

Let's map them to the full language name:

```r
movies_final <- movies_final %>%
  mutate(
    original_language =
      ISO_639_2$Name[
        match(original_language, ISO_639_2$Alpha_2)
      ]
  )

movies_final %>%
  distinct(original_language) %>%
  head(n = 5) %>%
  kable()
```

original_language
English
French
Chinese
Italian
Persian

Much better. Let's rename a few columns to be more accurate as well, and add a `year` column, which will be useful for EDA

```r
movies_final <- movies_final %>%
  rename(
    runtime_minutes = runtime,
    tmdb_vote_average = vote_average,
    tmdb_vote_count = vote_count,
    tmdb_popularity = popularity
  ) %>%
  mutate(release_year = year(release_date))
```

Removing Data

Before cleaning the data further, let's try understand our dataset a bit better. We should check our data's dimensions and remove any duplicated rows by ID:

```r
dataset_summary <- tibble(
  observations = nrow(movies_final),
  variables = ncol(movies_final),
  duplicate_rows = sum(duplicated(movies_final)),
  unique_movies = n_distinct(movies_final$id)
)

dataset_summary %>%
  kable(caption = "Dataset Summary")
```

Table 1: Dataset Summary

observations	variables	duplicate_rows	unique_movies
46632	25	1172	45434

```
movies_final <- movies_final %>%
  distinct(id, .keep_all = TRUE)
```

So we have 45,434 distinct films and 24 distinct variables. Let's now take a look at missingness:

```
missing_summary <- movies_final %>%
  summarise(across(everything(), ~ mean(is.na(.)) * 100)) %>%
  pivot_longer(
    cols = everything(),
    names_to = "column",
    values_to = "missing_pct"
  ) %>%
  arrange(desc(missing_pct))

missing_summary %>%
  kable(
    digits = 2,
    caption = "Percentage of Missing Values by Column"
  )
```

Table 2: Percentage of Missing Values by Column

column	missing_pct
belongs_to_collection	90.12
tagline	55.10
overview	2.11
avg_user_rating	1.59
num_user_ratings	1.59
original_language	0.77
runtime_minutes	0.57
release_date	0.19
release_year	0.19
imdb_id	0.04
tmdb_popularity	0.01
revenue	0.01
spoken_languages	0.01
title	0.01
tmdb_vote_average	0.01
tmdb_vote_count	0.01
production_companies	0.01
production_countries	0.01
keywords	0.00
cast	0.00
crew	0.00
adult	0.00
budget	0.00

column	missing_pct
id	0.00
genres	0.00

In the grand scheme of things, this is a pretty clean dataset! Let's go column by column. `belongs_to_collection` can easily be removed. Not every film is part of a collection, and while the first and second films in a collection may be closely related to each other and may be the best recommendation, we aim to achieve this similarity using other features, such as cast and film name. We will impute `tagline` and replace with an empty string, which will mean that there is no tagline. A tagline can have useful information, and missingness is too high (55%) for imputing. We will do the same for `overview`, even if there's only 2% missing.

Numeric columns can be imputed by using the median, and we will do so for `avg_user_rating`, `num_user_ratings`, and `runtime`. The other rows (`release_date`, `imdb_id`, `original_language`, and all others) all have missingness less than 4%, so we'd be removing very few rows overall.

```
movies_clean <- movies_final %>%
  select(-belongs_to_collection) %>%
    mutate(
    tagline = replace_na(tagline, ""),
    overview = replace_na(overview, "")
  ) %>%
    mutate(
    avg_user_rating = replace_na(avg_user_rating, median(avg_user_rating, na.rm = TRUE)),
    num_user_ratings = replace_na(num_user_ratings, median(num_user_ratings, na.rm = TRUE)),
    runtime_minutes = replace_na(runtime_minutes, median(runtime_minutes, na.rm = TRUE))
  ) %>%
    drop_na(
    release_date,
    imdb_id,
    original_language,
    tmdb_popularity,
    revenue,
    spoken_languages,
    title,
    tmdb_vote_average,
    tmdb_vote_count,
    production_companies,
    production_countries,
    cast,
    crew,
    adult,
    budget,
    id,
    genres
  )

sum(is.na(movies_clean)) %>% kable()
```

x
0

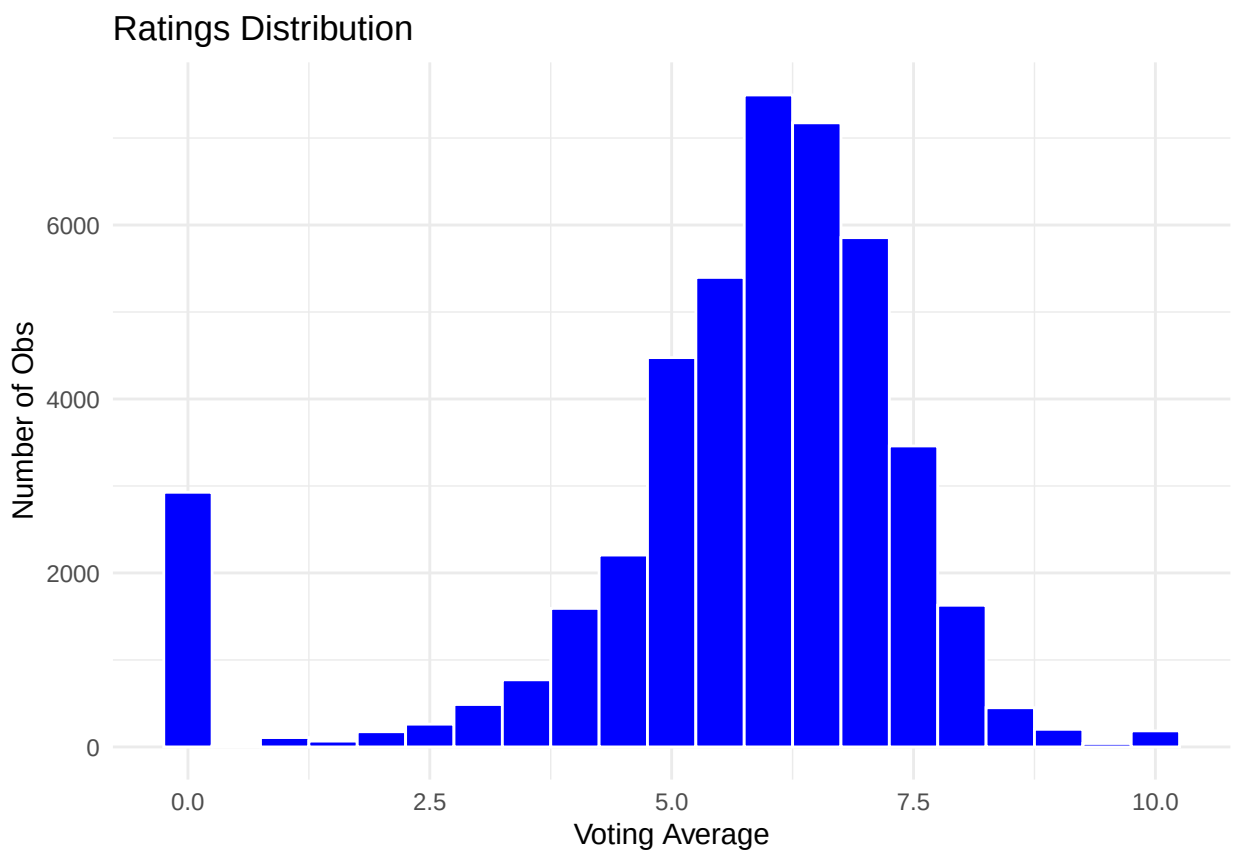
Now we are left with 113 films less, and can confidently start our EDA.

EDA

Distribution of Ratings

Let's take a look the distribution of ratings:

```
ggplot(movies_clean, aes(tmdb_vote_average)) +
  geom_histogram(binwidth=0.5, fill = "blue", color = "white") +
  labs(
    title = "Ratings Distribution",
    x = "Voting Average",
    y = "Number of Obs"
  ) +
  theme_minimal()
```

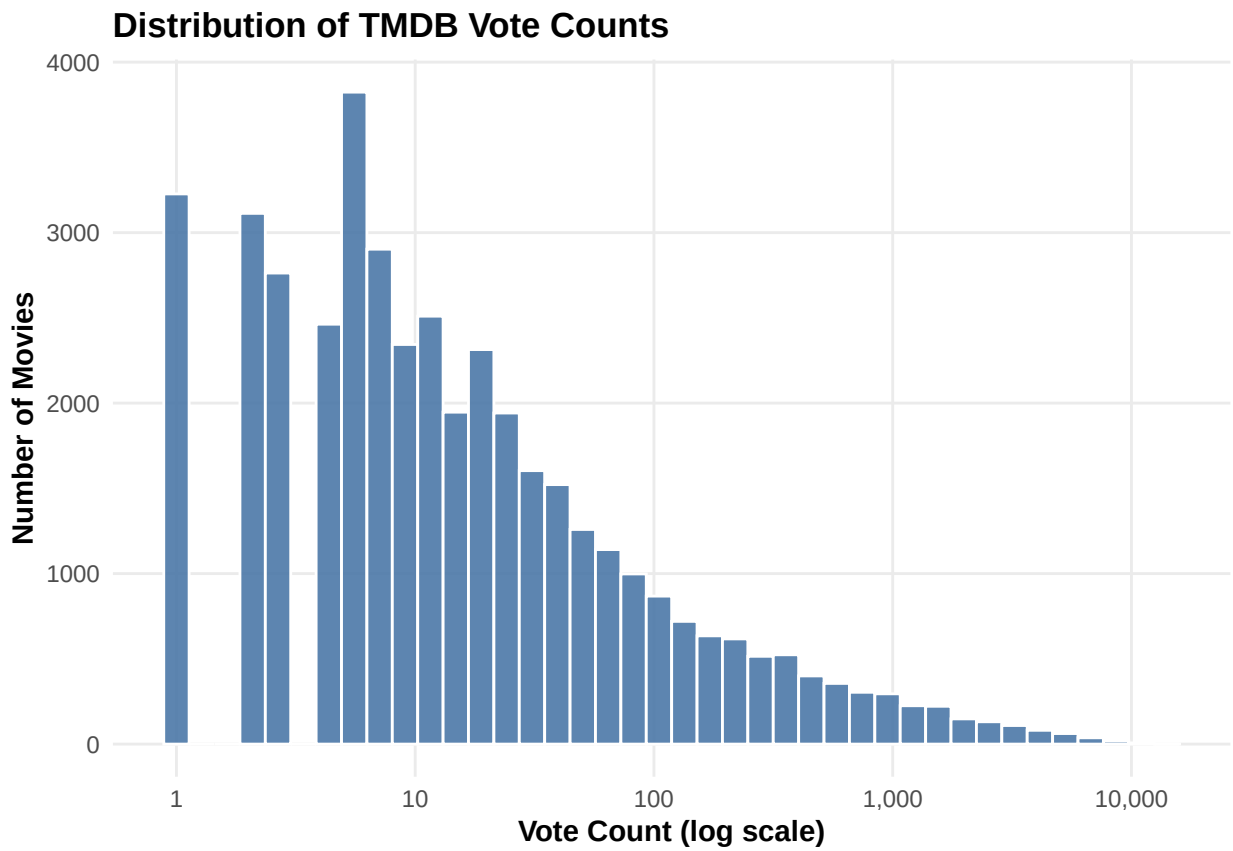


This is expected; users seem to be more likely to either give a "slightly above average" rating, or a complete dislike, i,e, 0. Few usually give full marks. So the data is slightly skewed to the left.

Ratings Count

Let's also take a look at ratings count - it's expected that there's most likely some correlation with more popular films, i.e., films with hundreds of millions for a budget or films that are generally rated higher

```r
ggplot(movies_clean, aes(x = tmdb_vote_count)) +
  geom_histogram(
    bins = 40,
    fill = "#4C78A8",
    color = "white",
    alpha = 0.9
  ) +
  scale_x_log10(labels = scales::comma) +
  labs(
    title = "Distribution of TMDB Vote Counts",
    x = "Vote Count (log scale)",
    y = "Number of Movies"
  ) +
  theme_minimal() +
  theme(
    plot.title = element_text(face = "bold"),
    plot.subtitle = element_text(color = "gray40"),
    axis.title = element_text(face = "bold"),
    panel.grid.minor = element_blank()
  )
```

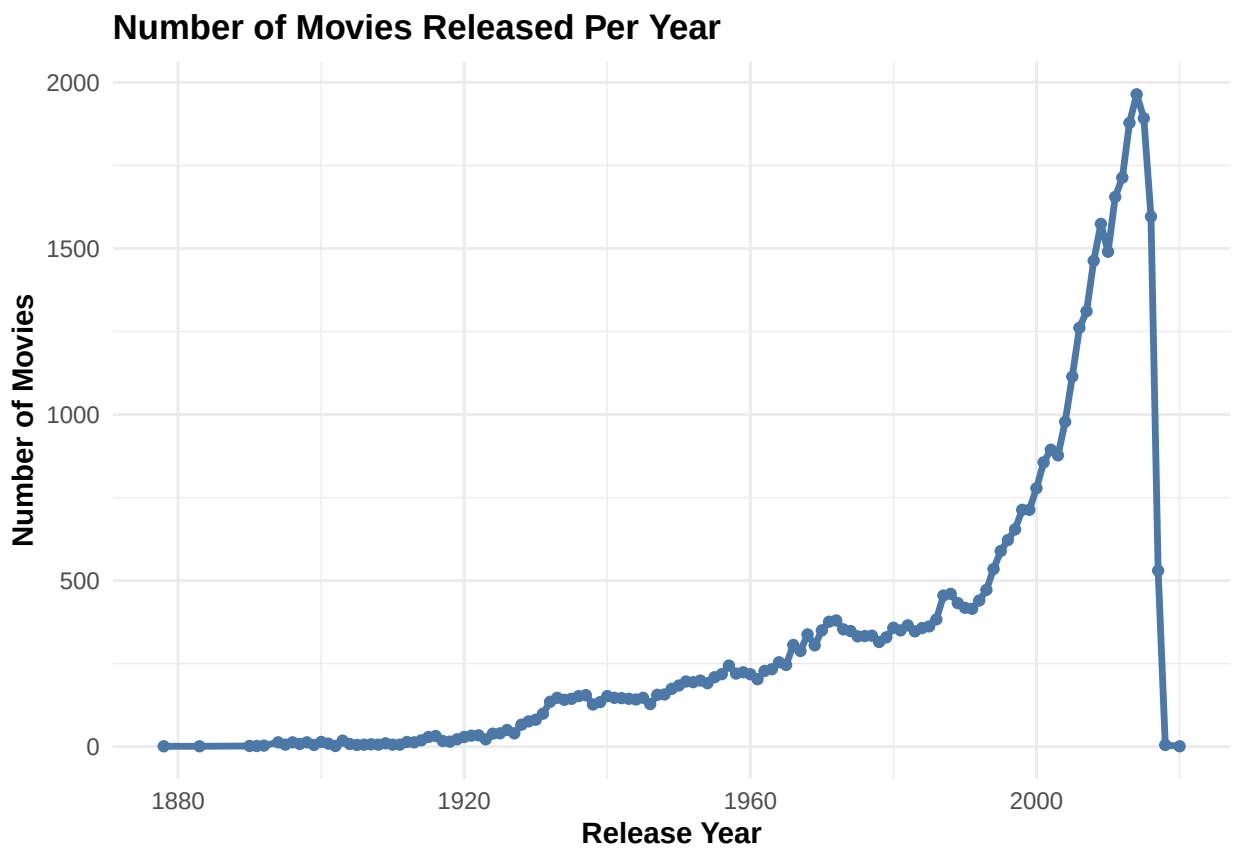


We see a long tail, with the data skewed to the right. This again, does make sense - most films don't receive many ratings. Think of how many blockbuster films come out per year relative to all "other" films. This most most likely can be seen in any data that has popularity attached to it, and the movie industry is no different.

Movies by Year

Let's take a look at comparing movies and their release date. It's likely that more films were released in recent times. It would be interesting to have included more recent data, due to COVID-19 and recent strikes in the film industry. Still, let's see if there are any visible trends:

```
movies_clean %>%
  count(release_year) %>%
  ggplot(aes(x = release_year, y = n)) +
  geom_line(
    color = "#4C78A8",
    linewidth = 1.2
  ) +
  geom_point(
    color = "#4C78A8",
    size = 1.5
  ) +
  labs(
    title = "Number of Movies Released Per Year",
    x = "Release Year",
    y = "Number of Movies"
  ) +
  theme_minimal() +
  theme(
    plot.title = element_text(face = "bold"),
    plot.subtitle = element_text(color = "gray40"),
    axis.title = element_text(face = "bold"))
```



What's going on here? We see what looks to be an exponential increase, which does make sense (growth of industry, technological advances for animation and CGI), but we see a sudden drop towards the end of the data. Most likely, there's entries about films which would be released after 2017 (the cutoff of the data, so there aren't many datapoints for later dates). Let's take a closer look:

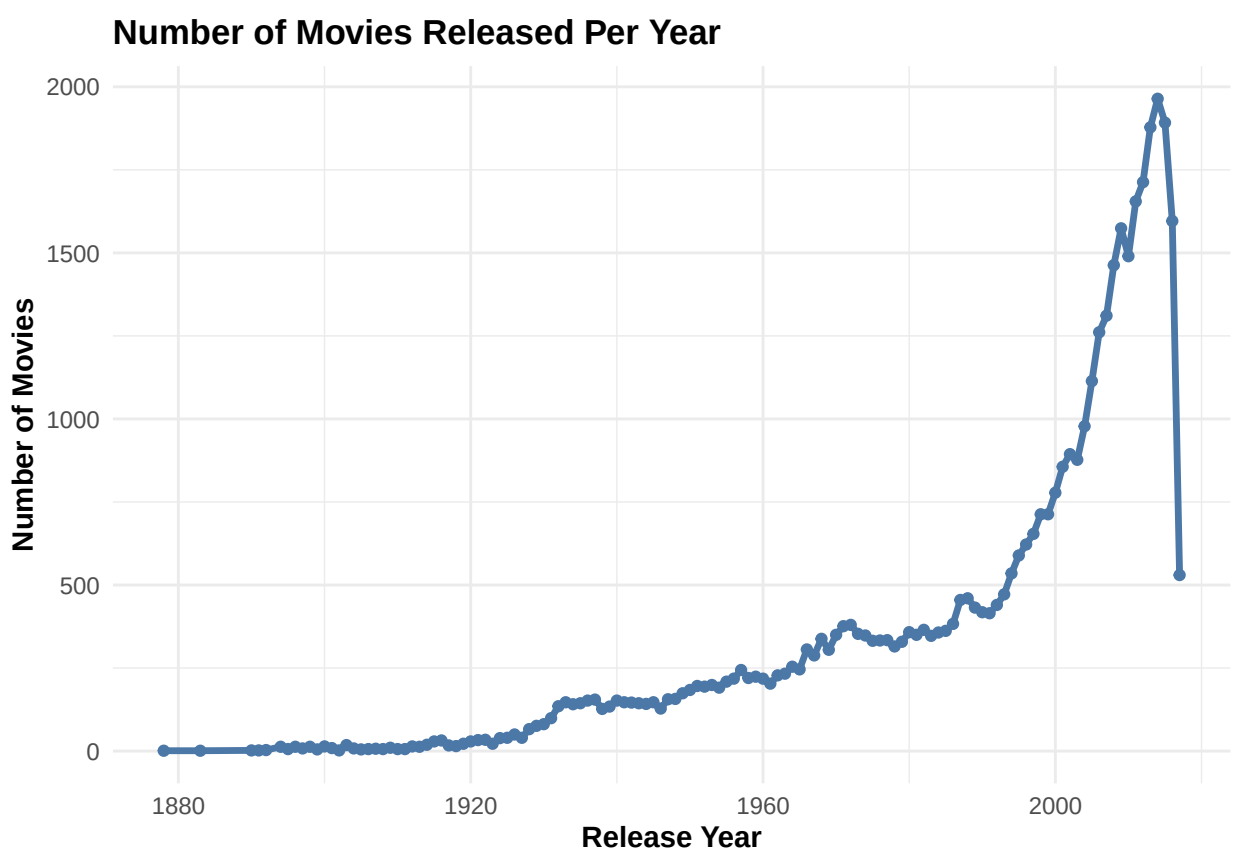
```
movies_clean %>%
  filter(release_year > 2015) %>%
  select(title, tmdb_vote_count, release_year) %>%
  count(release_year, sort = TRUE) %>%
  kable()
```

release_year	n
2016	1596
2017	530
2018	5
2020	1

So we see a few things happening. First, reviewers had left reviews for unreleased films - they are time travelers! Let's remove these films from the data. Second, as this dataset has a cutoff of July 2017, according to its description, so about half of the films released in 2017 are missing, which explains the drop.

```
movies_clean <- movies_clean %>%
  filter(release_year <= 2017)
```

```r
movies_clean %>%
  count(release_year) %>%
  ggplot(aes(x = release_year, y = n)) +
  geom_line(
    color = "#4C78A8",
    linewidth = 1.2
  ) +
  geom_point(
    color = "#4C78A8",
    size = 1.5
  ) +
  labs(
    title = "Number of Movies Released Per Year",
    x = "Release Year",
    y = "Number of Movies"
  ) +
  theme_minimal() +
  theme(
    plot.title = element_text(face = "bold"),
    plot.subtitle = element_text(color = "gray40"),
    axis.title = element_text(face = "bold"))
```



Genres

Genre vs. Rating

Word Cloud

keywords_tokens %>% count(word, sort=TRUE) bind_tf_idf(word, genre, n)

Numeric Features Heatmap

corrplot(cor(numeric_df))

Runtime Distribution

Popularity vs. Rating

Bigrams

use tidytext, lab 8

Cosine Similarity, lab 8